

Index Key Columns vs Included Columns

#sql-server #performance #indexing #physical-design

The Two Parts of a Nonclustered Index

A nonclustered index in SQL Server has two distinct sections:

1. Key columns: the columns that define the b-tree's sort order. These columns appear at every level of the tree, from root pages down to leaf pages.
2. Included columns: extra columns that appear only at the leaf level. They are not part of the sort order and do not affect navigation through the tree.

```
Create Index IX_0eOrder_StatusDate
On 0eOrder (OrderStatus, DateFilled)      -- key columns
Include (OrderId, GroupNum, PriInternal)  -- included columns
```

The key columns are how the index is searched. The included columns are along for the ride, available to satisfy the query's column list without forcing a key lookup back to the clustered index.

Why the Distinction Matters

The b-tree only needs to sort by the columns the optimizer will actually use to navigate. Every column you add to the key list makes the tree wider at every level, which means more pages, more memory, and slower writes. Included columns sit at the leaf only, so they cost storage but not navigation overhead.

A useful mental model: key columns are the index's address book; included columns are the contents of the apartment at each address. Anyone looking someone up uses the address book. Once they arrive, the contents are right there to read.

When a Column Belongs in the Key

A column should be a key column when at least one of these is true:

1. The optimizer will use it to seek (an equality or range predicate is applied to it).
2. The optimizer will use it to satisfy an ORDER BY without a separate Sort operator.
3. The optimizer will use it to satisfy a GROUP BY in stream aggregate fashion.
4. It contributes to uniqueness (you're declaring this index unique on the combination of these columns).

If none of those apply, the column is wasted in the key.

Key Order Matters

The leftmost key column drives selectivity. SQL Server can use a nonclustered index to seek on (A) and on (A, B) and on (A, B, C), but not on (B) alone or on (B, C) alone. The order of the keys defines which queries the index can serve.

The general rule: put the most-frequently-filtered, highest-selectivity column first. Equality predicates before range predicates. ORDER BY columns last unless they are also the filter columns.

When a Column Belongs in INCLUDE

A column should be in the INCLUDE list when:

1. The query's SELECT list or WHERE residual references the column, but the optimizer doesn't seek on it.

2. You need the column at the leaf for a covering index, but adding it to the key would slow down the tree without speeding up navigation.

The win for INCLUDE is avoiding the key lookup (sometimes called a bookmark lookup), which is the operation where the engine seeks the nonclustered index, gets a row pointer, and then has to seek the clustered index to fetch the columns the nonclustered index doesn't carry. Key lookups are nested-loop seeks against the clustered index, and they add up fast at scale.

Covering Indexes

An index “covers” a query when every column the query references (in SELECT, WHERE, JOIN, GROUP BY, and ORDER BY) is available in the index, either as a key or as an INCLUDE. The optimizer can satisfy the query entirely from the nonclustered index without touching the clustered index.

```
-- Query
Select OrderId, GroupNum, PriInternal
From OeOrder
Where OrderStatus = 'Ready To Ship'
And DateFilled >= '2026-01-01'

-- Covering index
Create Index IX_OeOrder_Ready
On OeOrder (OrderStatus, DateFilled)
Include (OrderId, GroupNum, PriInternal)
```

The optimizer seeks on (OrderStatus, DateFilled), finds matching rows at the leaf, and reads OrderId, GroupNum, PriInternal directly from the leaf entry. No key lookup, no clustered index access.

When a Covering Index Is Worth It

- The query is frequent and important.
- The columns in the SELECT list are stable (they don't churn often).
- The current plan shows a high cost on Key Lookup operators.
- The table is large enough that key lookups are measurably hurting throughput.

When It Isn't

- The query is rare or low-priority.
- The included columns include large strings, BLOBs, or LOB columns that bloat the leaf.
- Write throughput on the table is high. Every column you include in the index has to be maintained on every UPDATE that touches it.

Filtered Indexes

A filtered index has a WHERE clause in its definition. The index only contains rows matching the filter. This is appropriate when the optimizer's queries always include the same predicate, and the predicate matches a small minority of the table.

```
Create Index IX_ImgImage_Pending
On ImgImage (Id)
Include (ContentCode)
Where IsMovedToFile Is Null Or IsMovedToFile <> 1
```

The index is small (only the rows that aren't moved), so seeks are cheap and writes don't have to maintain entries for the rest of the table. Useful when:

- A specific subset of rows is queried far more often than the rest.
- The subset is a small fraction of the table.
- The filter expression is simple (the optimizer matches filtered indexes via expression equivalence, not arbitrary logic).

The catch: parameter-driven queries don't always match a filtered index. If the WHERE clause uses `IsMovedToFile = @flag`, the optimizer can't prove at compile time that the value matches the filter, so it won't use the index. Filtered indexes work best when the predicate is hard-coded in the query.

How to Read the Optimizer's Index Choice

When examining an actual execution plan, look at three things on each Index Seek or Index Scan operator:

1. The index name. Confirm the optimizer chose the index you expected. If a query that should use `IX_OrderStatus` is using `PK_Order` instead, that is a signal. Either the expected index doesn't exist, the optimizer thinks the PK is cheaper, or the query is shaped in a way that prevents the optimizer from considering it.
2. The Seek Predicates property. These are the columns the optimizer is actually using to navigate the b-tree. They should match the leftmost key columns of the chosen index.
3. The Predicate property (the residual). This is what is evaluated after the seek, against the rows the seek returned. A long residual filter on a Seek operator is a hint that you have a different index than you need. The optimizer found something it could seek on, but is filtering aggressively after the seek.

A long residual on a Seek is a common cause of high reads. The seek looks like it's doing the right thing on paper, but each leaf row is being checked against more predicates than it should. Adding a column to the key (to convert a residual into a seek predicate) or to the INCLUDE list (to keep the seek shape but cover the residual) are both possible fixes.

Diagnostic: Find Indexes That Could Be Smaller

This query finds nonclustered indexes where the trailing key columns are rarely used as seek predicates and could probably be moved to INCLUDE:

```
Select
    Object_Name(s.object_id) As TableName,
    i.name As IndexName,
    i.type_desc,
    s.user_seeks,
    s.user_scans,
    s.user_lookups,
    s.user_updates,
    Cast(Cast(s.user_updates As Float)
        / NullIf(s.user_seeks + s.user_scans + s.user_lookups + s.user_updates, 0)
        * 100 As Decimal(5,2)) As PctWrites
From sys.dm_db_index_usage_stats s
Join sys.indexes i On i.object_id = s.object_id And i.index_id = s.index_id
Where s.database_id = Db_Id()
    And i.type_desc = 'NONCLUSTERED'
Order By s.user_updates Desc
```

If an index has high `user_updates` and low `user_seeks`, it's costing more to maintain than it earns back. Either reshape it (move columns from key to INCLUDE) or drop it.

The OeOrderHistory Example

In the `lsp_imgGetListOfTopXImagesToMove` refactor, the win came down to which `OeOrderHistory` index the optimizer chose:

- The original `LEFT JOIN + OR` plan used `PK_OeOrderHistory`, which is keyed on `OrderId`. The optimizer sought by `OrderId` for each association row, then applied `OrderStatus = 'Shipped'` as a residual predicate. Each seek touched several pages to read the rows for that `OrderId`, and most of those rows were discarded because their status wasn't `Shipped`.
- The `materialize-once + EXISTS` plan used `ByOrderStatus`, an index keyed on `OrderStatus`. The optimizer could now seek directly to `Shipped` rows and probe associations from there. Per-table reads on `OeOrderHistory` dropped from 272,725 to 36,259, a 7x reduction.

The shape of the query mattered (the `EXISTS` lifted the `OrderStatus` filter into a place the optimizer could push it down to a seek predicate), but so did the existence of the right index. Without `ByOrderStatus`, no query rewrite would have produced the same win.

Related Concepts

- **FORCESEEK Hints**: forces the optimizer to seek on whatever index it has, but doesn't help if the index keys don't match the predicate.
- **Non-SARGable Predicates**: even with the right index, a function-wrapped column can't be sought on.
- **Density Vector**: how the optimizer estimates rows when it can seek on a key column but doesn't know the value.
- **Query Store Triage**: high-read plans often reveal themselves as Index Scan or long-residual Index Seek operators.